

Universidad del Valle de Guatemala

Facultad de Ingenieria

Departamento de Ciencias de la Computacion

CC3091 - Ingenieria de Software 2

Semestre II - 2026

Tarea 3

Pruebas Automatizadas

Proyecto:

App de Apoyo y Orientacion para Solicitantes de Visa

Integrantes:

Diego Quan - 24336

Diego Sebastian Guevara Casasola - 24128

Juan Francisco Orozco Mijangos - 24647

Norman Aguirre - 24479

Ciudad de Guatemala, Guatemala

Julio de 2026

Contenido

1. Resumen ejecutivo
2. Introduccion
3. Descripcion del proyecto
4. Arquitectura real del sistema
5. Importancia de las pruebas automatizadas
6. Fundamentos de pruebas
7. Herramientas investigadas para frontend
8. Herramientas investigadas para backend
9. Comparacion de herramientas
10. Seleccion de herramientas
11. Justificacion de Vitest y React Testing Library
12. Justificacion de Jest y Supertest
13. Configuracion implementada
14. Estrategia de pruebas
15. Pruebas implementadas en frontend
16. Pruebas implementadas en backend
17. Resultados
18. Cobertura
19. Ventajas
20. Desventajas
21. Tiempo empleado
22. Hallazgos tecnicos
23. Conclusiones
24. Recomendaciones
25. Referencias
26. Anexos

1. Resumen ejecutivo

Este informe presenta la investigacion, seleccion, configuracion e implementacion de pruebas automatizadas para la App de Apoyo y Orientacion para Solicitantes de Visa. El sistema analizado contiene un frontend construido con React 19.2.4 y Vite 8.0.1, y un backend Node.js con Express 5.2.1, PostgreSQL, `multer` y Cloudflare R2 mediante `@aws-sdk/client-s3`. La tarea se enfoco en identificar herramientas compatibles con la arquitectura real del repositorio, sin inventar componentes ni endpoints. Para frontend se seleccionaron Vitest, React Testing Library, `@testing-library/user-event`, `@testing-library/jest-dom`, jsdom y cobertura V8. Para backend se seleccionaron Jest y Supertest con mocks de `pg.Pool` y R2. Se implementaron 12 pruebas frontend y 14 pruebas backend. La ejecucion local reporto 12/12 pruebas frontend aprobadas y 14/14 pruebas backend aprobadas. La cobertura frontend fue 65.95% statements, 48.40% branches, 75.29% functions y 68.91% lines. La cobertura backend fue 36.81% statements, 20.75% branches, 50.00% functions y 37.55% lines.

2. Introduccion

Las pruebas automatizadas permiten verificar de forma repetible que un sistema conserva su comportamiento esperado despues de cambios de codigo. En una aplicacion de apoyo a solicitantes de visa, la calidad del software es especialmente relevante porque la plataforma administra orientacion, sesiones, formularios, documentos y seguimiento de tramites. El objetivo del informe es documentar las herramientas investigadas, justificar la seleccion tecnica y presentar pruebas implementadas sobre archivos reales del repositorio.

3. Descripcion del proyecto

El proyecto busca orientar a personas que desean solicitar o renovar visas, especialmente estadounidense y mexicana. El frontend contiene pantallas para onboarding, login, seleccion de perfil, dashboard, formulario DS-160, documentos, entrevista, notificaciones, chat y perfil. El backend expone endpoints REST para usuarios, tramites, documentos, DS-160, banco de preguntas, notificaciones y sesiones de entrevista. La base de datos PostgreSQL se define inicialmente en `init.sql`.

Tecnologia	Archivo o ubicacion	Funcion
React	<code>frontend/src/App.jsx</code>	Interfaz de usuario y rutas
Vite	<code>frontend/vite.config.js</code>	Build, dev server y pruebas frontend
React Router DOM	<code>frontend/src/App.jsx</code>	Navegacion entre pantallas
Express	<code>backend/app.js</code>	API REST
PostgreSQL	<code>init.sql</code>	Persistencia de usuarios, tramites, documentos y sesiones
pg	<code>backend/app.js</code>	Conexion a PostgreSQL
multer	<code>backend/upload.js</code>	Manejo de archivos en memoria
Cloudflare R2/S3 SDK	<code>backend/r2.js</code>	Almacenamiento externo de archivos
Docker Compose	<code>docker-compose.yml</code>	Orquestacion de frontend, backend y base

4. Arquitectura real del sistema

La aplicación usa una arquitectura cliente-servidor. El frontend React consume el backend mediante `fetch` y la función `buildApiUrl` ubicada en `frontend/src/config/api.js`. El backend Express registra middlewares CORS, `express.json()` y rutas reales. La persistencia se realiza en PostgreSQL mediante `Pool` de `pg`. Para documentos y audios se usa `multer` y funciones de `R2` en `backend/r2.js`.

Refactor implementado:

- `backend/app.js`: contiene la configuración de Express, middlewares, conexión, rutas y exporta `app`.
- `backend/index.js`: importa `app` y ejecuta `app.listen`.

Este cambio permite que Supertest importe `app` sin abrir un puerto durante las pruebas.

5. Importancia de las pruebas automatizadas

En este proyecto, las pruebas automatizadas reducen el riesgo de romper flujos de alto impacto: login, registro, validación de sesión, banco de preguntas, notificaciones, carga de documentos y entrevista. También permiten simular fallos de base de datos sin afectar datos reales.

6. Fundamentos de pruebas

Concepto	Aplicación al proyecto
Prueba unitaria	<code>buildApiUrl</code> , hooks y componentes aislados
Prueba de componente	<code>QuestionBankModal</code> , <code>QuestionBankCard</code> , <code>DocumentCard</code> , <code>ProfileSelection</code>
Prueba de integración HTTP	Endpoints Express con Supertest y mocks
Prueba E2E	No implementada; sería candidata con Playwright o Cypress
Mock	<code>fetch</code> , <code>pg.Pool</code> , <code>R2</code>
Stub	Respuestas fijas para consultas SQL
Spy	Verificación de llamadas como <code>fetch</code> y callbacks
Assertion	<code>expect</code> , <code>toBe</code> , <code>toEqual</code> , <code>toHaveBeenCalledWith</code>
Cobertura	Reportes V8 y Jest para statements, branches, functions y lines

7. Herramientas investigadas para frontend

Herramienta	Propósito	Ventajas	Desventajas	Integración
Vitest	Runner unitario para Vite	Reusa configuración Vite, rápido, soporta JSX y V8 coverage	Requiere configurar jsdom para DOM	Muy alta
Jest	Runner general	Popular, robusto, buena documentación	En Vite puede requerir más transformación/configuración	Media

Herramienta	Proposito	Ventajas	Desventajas	Integracion
React Testing Library	Pruebas de comportamiento visible	Enfoca pruebas en DOM y uso real	No prueba detalles internos	Alta
Cypress	Component/E2E	Excelente para flujos visuales en navegador	No es herramienta puramente unitaria	Futura
Playwright	E2E y componentes	Multi-navegador, fuerte para flujos reales	Mayor costo de ejecucion	Futura

8. Herramientas investigadas para backend

Herramienta	Proposito	Ventajas	Desventajas	Integracion
Jest	Runner y mocks	Encaja con CommonJS, mocks integrados	Puede requerir <code>--runInBand</code> cuando hay estado global	Alta
Vitest	Runner moderno	Rapido y compatible con Jest APIs	Backend actual no usa Vite ni ESM	Media
Mocha	Runner flexible	Maduro, configurable	Requiere combinar con Chai/Sinon para assertions/mocks	Media
Chai	Assertions	Sintaxis expresiva	No es runner por si solo	Complementaria
Supertest	HTTP assertions	Prueba Express sin puerto fijo	No reemplaza mocks de BD	Alta
Node.js Test Runner	Runner nativo	Sin dependencia externa	Menos ecosistema de mocks que Jest	Media
Sinon	Spies/stubs/mocks	Muy robusto	Redundante si Jest ya cubre mocks necesarios	Opcional

9. Comparacion de herramientas

Criterio	Vitest + RTL	Jest frontend	Jest + Supertest	Mocha/Chai/Sinon	Node Test Runner
Compatibilidad React/Vite	Alta	Media	No aplica	Media	Media
Compatibilidad Express/CommonJS	Media	Alta	Alta	Alta	Alta
Facilidad de configuracion	Alta	Media	Alta	Media	Media
Comunidad	Alta	Alta	Alta	Alta	Alta
Documentacion	Alta	Alta	Alta	Alta	Alta
Mocks	Alta	Alta	Alta	Alta	Media
Cobertura	Alta	Alta	Alta	Con nyc/c8	Nativa/externa
Curva de aprendizaje	Baja	Baja	Baja	Media	Media

Criterio	Vitest + RTL	Jest frontend	Jest + Supertest	Mocha/Chai/Sinon	Node Test Runner
Mantenimiento	Alto	Alto	Alto	Alto	Alto

10. Seleccion de herramientas

Frontend: Vitest + React Testing Library + user-event + jest-dom + jsdom + V8 coverage.

Backend: Jest + Supertest + mocks de `pg.Pool` y `R2`.

11. Justificacion de Vitest y React Testing Library

Vitest es adecuado porque el frontend usa Vite. La configuracion se agrego directamente en `frontend/vite.config.js`, con `environment: "jsdom"`, `globals: true`, `setupFiles` y cobertura V8. React Testing Library permite comprobar lo que el usuario ve y manipula: botones, formularios, mensajes y cambios de estado.

12. Justificacion de Jest y Supertest

El backend usa CommonJS. Jest permite mocks de modulos como `pg` y `../r2` sin convertir el proyecto a ES Modules. Supertest permite enviar requests a `app` directamente, sin ejecutar `app.listen` ni abrir puertos.

13. Configuracion implementada

Frontend:

- `frontend/package.json`: scripts `test`, `test:run`, `test:coverage`, `test:watch`.
- `frontend/vite.config.js`: configuracion Vitest.
- `frontend/src/setupTests.js`: `jest-dom`, limpieza de mocks y storage.

Backend:

- `backend/app.js`: exporta la aplicacion Express.
- `backend/index.js`: arranca el servidor.
- `backend/jest.config.js`: ambiente Node y cobertura.
- `backend/__tests__/app.test.js`: pruebas HTTP.

14. Estrategia de pruebas

La matriz completa se encuentra en `docs/estrategia-pruebas.md`. La suite implementada prioriza pruebas de alto valor: autenticacion, configuracion API, tema, seleccion de perfil, componentes administrativos, documentos, registro, login, banco de preguntas, notificaciones y entrevista.

15. Pruebas implementadas en frontend

Archivo de prueba	Archivo probado	Escenario
frontend/src/__tests__/api.test.js	frontend/src/config/api.js	Normalizacion de API base y rutas
frontend/src/__tests__/useRequireAuth.test.jsx	frontend/src/hooks/useRequireAuth.jsx	Sesion valida e invalida
frontend/src/__tests__/useTheme.test.jsx	frontend/src/hooks/useTheme.jsx	Cambio de tema
frontend/src/__tests__/QuestionBankModal.test.jsx	frontend/src/components/QuestionBankModal.jsx	Crear y editar pregunta
frontend/src/__tests__/QuestionBankCard.test.jsx	frontend/src/components/QuestionBankCard.jsx	Editar y eliminar
frontend/src/__tests__/DocumentCard.test.jsx	frontend/src/components/DocumentCard.jsx	Validacion de formato y usuario
frontend/src/__tests__/ProfileSelection.test.jsx	frontend/src/pages/ProfileSelection/ProfileSelection.jsx	Seleccion y guardado de perfil

Fragmento representativo:

```
await user.click(screen.getByText("Turismo / Negocios (B1/B2)"));
await user.click(screen.getByRole("button", { name: "Continuar" }));
expect(localStorage.getItem("perfilUsuario")).toBe("turismo_negocios");
```

16. Pruebas implementadas en backend

Endpoint	Escenarios
GET /validar-sesion	Correo existente y no existente
POST /register	Registro correcto y error de BD
POST /login	Credenciales validas e incorrectas
GET /questions	Listado simulado
POST /questions	Creacion valida
PUT /questions/:id	Actualizacion
DELETE /questions/:id	Eliminacion
GET /notificaciones/:userId	Listado
PUT /notificaciones/:id/leer	Marcar leida
POST /interview-sessions	Crear sesion
PUT /interview-sessions/:id/feedback	Guardar feedback

Fragmento representativo:

```
const response = await request(app)
  .post("/login")
  .send({ correo: "login@example.com", contrasena: "1234" });
expect(response.status).toBe(200);
expect(response.body.message).toBe("Login exitoso");
```

17. Resultados

Metrica	Frontend	Backend
Pruebas ejecutadas	12	14
Pruebas aprobadas	12	14
Pruebas fallidas	0	0
Archivos de prueba	7	1
Tiempo suite sin cobertura	3.08 s	1.225 s
Tiempo suite con cobertura	3.91 s	2.354 s

Nota: durante frontend aparece el aviso de jsdom `Not implemented: navigation to another Document`, originado por rutas que asignan `window.location.href`; no provoco fallo.

18. Cobertura

Capa	Statements	Branches	Functions	Lines
Frontend	65.95%	48.40%	75.29%	68.91%
Backend	36.81%	20.75%	50.00%	37.55%

Los valores deberan sustituirse por los resultados generados al ejecutar la suite de pruebas en el entorno local del equipo si se modifica el codigo o se agregan nuevas pruebas.

19. Ventajas

- Integracion natural con Vite en frontend.
- Pruebas centradas en comportamiento visible.
- Backend probado sin abrir puertos.
- PostgreSQL y R2 completamente simulados.
- Separacion clara entre `app.js` e `index.js`.
- Suite rapida para ejecucion local.

20. Desventajas

- Cobertura backend inicial baja por cantidad de endpoints en `app.js`.
- Algunas pantallas requieren mocks de `localStorage` y navegacion.
- `app.js` conserva mucha logica centralizada, lo que reduce granularidad.
- No hay pruebas E2E todavia.

21. Tiempo empleado

Actividad	Tiempo estimado	Tiempo real	Observaciones
Analisis del repositorio	45 min	Pendiente de registrar por equipo	Realizado con lectura de archivos

Actividad	Tiempo estimado	Tiempo real	Observaciones
Configuracion frontend	35 min	Pendiente de registrar por equipo	Vitest y Testing Library
Configuracion backend	40 min	Pendiente de registrar por equipo	Jest y Supertest
Implementacion pruebas frontend	90 min	Pendiente de registrar por equipo	12 pruebas
Implementacion pruebas backend	90 min	Pendiente de registrar por equipo	14 pruebas
Ejecucion y ajustes	40 min	Pendiente de registrar por equipo	Suites verdes

22. Hallazgos tecnicos

- El backend necesitaba separar `app` de `app.listen`.
- `backend/app.js` ejecuta preparacion de esquemas al importarse; los mocks deben considerar esa inicializacion.
- `frontend/src/pages/ds160.jsx` contiene validaciones internas no exportadas.
- `npm install` reporto vulnerabilidades de auditoria: 7 frontend y 4 backend.
- Se observo mojibake en salidas de terminal, pero no se modificaron textos fuente.

23. Conclusiones

1. Vitest es la herramienta mas adecuada para el frontend por su integracion directa con Vite.
2. React Testing Library permite validar comportamiento visible sin acoplarse a detalles internos.
3. Jest funciona bien con el backend CommonJS existente.
4. Supertest permite probar Express sin iniciar un servidor real.
5. Los mocks de `pg.Pool` y `R2` reducen riesgos porque evitan tocar datos o archivos reales.
6. La cobertura inicial es util, pero debe ampliarse especialmente en backend.
7. La separacion `app.js/index.js` mejora la testabilidad sin cambiar endpoints.
8. La suite implementada sirve como base mantenible para agregar pruebas futuras.

24. Recomendaciones

- Agregar pruebas para `DS160Form` extrayendo validadores a un modulo utilitario.
- Separar endpoints grandes de `backend/app.js` hacia rutas/controladores.
- Agregar pruebas para upload real simulado de documentos.
- Incorporar Playwright o Cypress para flujos E2E.
- Revisar vulnerabilidades con `npm audit` antes de entrega final.
- Agregar la suite a CI cuando el repositorio tenga pipeline.

25. Referencias

Jest. (2026). *Jest: Delightful JavaScript Testing*. <https://jestjs.io/>

Node.js. (2026). *Test runner*. <https://nodejs.org/api/test.html>

React Testing Library. (2026). *React Testing Library*.
<https://testing-library.com/docs/react-testing-library/intro/>

Sinon.JS. (2026). *Getting started*. <https://sinonjs.org/getting-started/>

Supertest. (2026). *supertest*. <https://www.npmjs.com/package/supertest>

Vitest. (2026). *Next generation testing framework*. <https://vitest.dev/>

Vitest. (2026). *Coverage*. <https://vitest.dev/guide/coverage.html>

26. Anexos

Anexo A: estrategia completa en [docs/estrategia-pruebas.md](#).

Anexo B: pruebas frontend en [frontend/src/__tests__](#).

Anexo C: pruebas backend en [backend/__tests__/app.test.js](#).

Anexo D: capturas requeridas:

- [INSERTAR CAPTURA REAL DEL CODIGO:
[frontend/src/__tests__/ProfileSelection.test.jsx](#)]
- [INSERTAR CAPTURA REAL DEL CODIGO: [backend/__tests__/app.test.js](#)]
- [INSERTAR CAPTURA REAL DE LA TERMINAL: `npm run test:coverage` en frontend]
- [INSERTAR CAPTURA REAL DE LA TERMINAL: `npm run test:coverage` en backend]